

DALAS Project

Drug Repurposing

Vladislav Antipin (21242244)

Paul Béglin (21408798)

Master of Computer Science, Sorbonne University

January 8, 2026

Contents

1	Introduction	3
1.1	Problem Statement	3
1.2	Motivation and Objectives	3
1.3	Pipeline Overview	3
2	Data	5
2.1	Data Sources	5
2.2	Target variable definition	6
2.3	Data Quality and Preprocessing	7
2.4	Feature Description	9
3	Methods	11
3.1	Model Overview	11
3.1.1	Instance-Based Methods	11
3.1.2	Linear Models	11
3.1.3	Ensemble Methods	12
3.1.4	Model Selection Rationale	12
3.2	Implementation Details	13
3.2.1	Preprocessing Pipeline	13
3.2.2	Handling Class Imbalance	13
3.2.3	Train-Test Split Strategy	13
3.2.4	Cross-Validation and Hyperparameter Tuning	14
3.2.5	Software and Libraries	14
3.2.6	Evaluation Metrics	14
4	Results	15
4.1	Model Comparison	15
4.2	Feature Analysis	15
4.3	Failure and Success Cases	15
5	Discussion	15

1 Introduction

1.1 Problem Statement

Clearly define the problem your project addresses. Explain its relevance in the scientific or practical context.

The discovery of new drugs is, although crucial, an extremely expensive and time-consuming process. Many candidate drugs ultimately turn out to be not only ineffective, but also potentially toxic. It is thus of interest to repurpose already approved drugs for diseases they have not yet been tested against. To prioritize potential clinical trials, in addition to experts' knowledge, it's important to be able to predict the likelihood of success for such studies. This project aims to explore the data and develop a potential prediction pipeline for this purpose. We acknowledge that industrial and academic drug development employ far more comprehensive methods, this project, by contrast, focuses on a preliminary, data-driven approach and is exploratory in nature.

1.2 Motivation and Objectives

Explain why this problem is important. State your main objectives and hypotheses.

The ability to predict the success of drug repurposing has academic and clinical value, it helps reducing the experimental cost and accelerating the discovery of effective treatments. The main objectives of this project are to:

- Explore public datasets to identify some relevant features
- Develop a preliminary machine learning pipeline for success prediction
- Evaluate the model performance
- Identify biases and potential ways for improvement

We decided to specify a group of diverse but closely related diseases - immune system disorders and drugs that are approved for at least one member of this group.

1.3 Pipeline Overview

To address the drug repurposing prediction problem, we developed a data pipeline that integrates multiple public biomedical databases, extracts relevant features, and trains machine learning models. Figures 1, 2, and 3 provide increasingly detailed views of this pipeline.

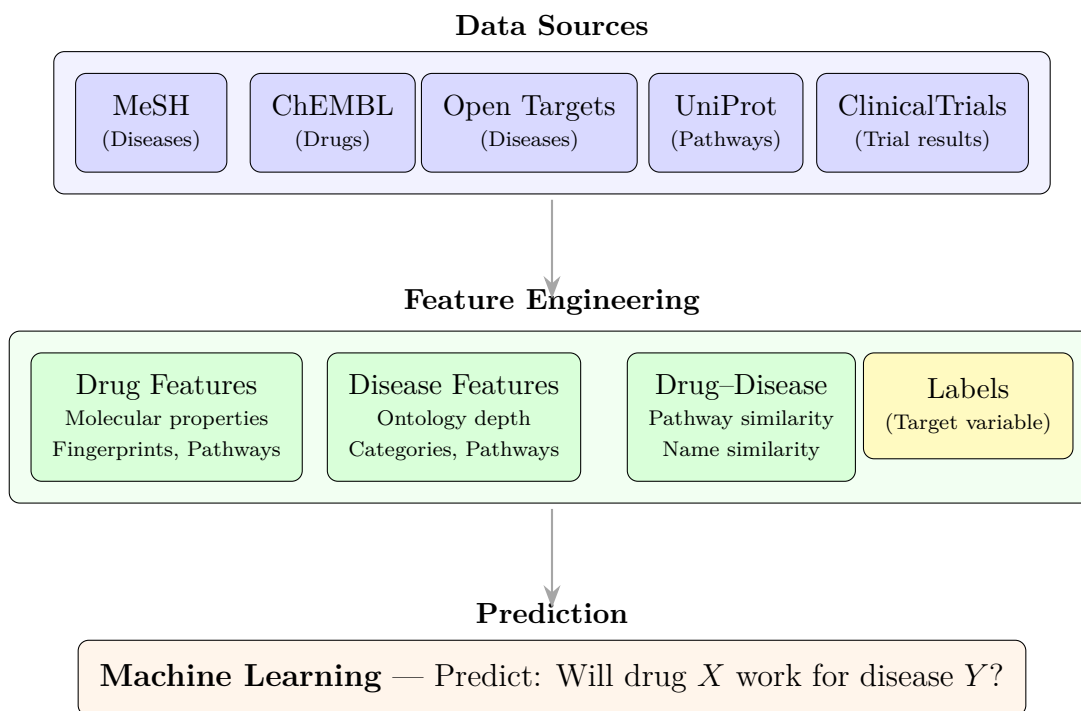


Figure 1: Simplified pipeline overview showing the three main stages of our drug repurposing prediction system. Data from multiple biomedical databases is processed into numerical features, which are then used to train machine learning models. The Labels box (yellow) represents the target variable—trial success or failure—derived from clinical trial outcomes using heuristic rules we defined.

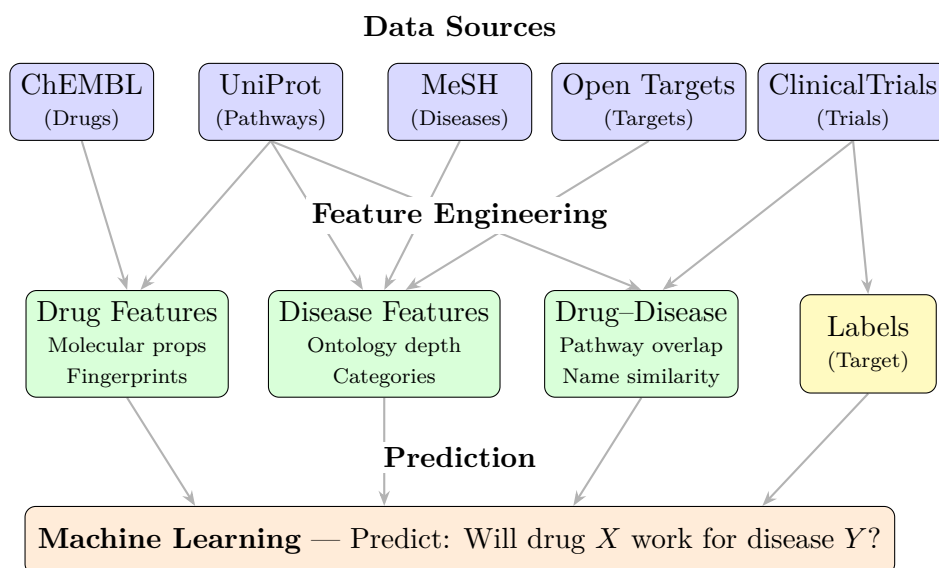


Figure 2: Same as Figure ?? but with labels having white backgrounds to create visual “gaps” where arrows pass behind text.

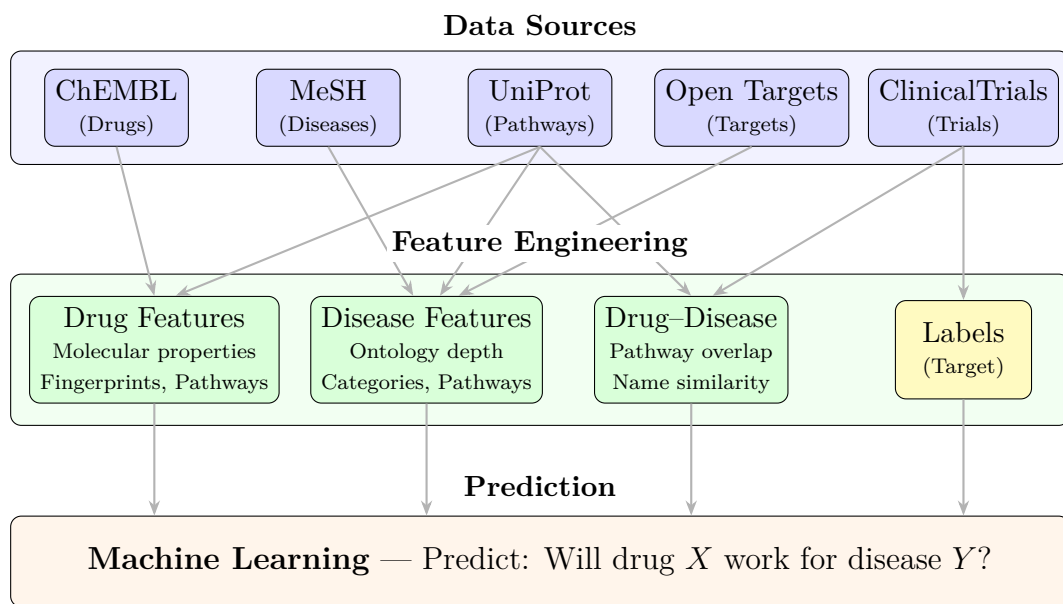


Figure 3: Complete pipeline overview for drug repurposing prediction. Five public biomedical databases provide the raw data, which is transformed into three feature categories plus labels. The yellow Labels box is distinguished because it represents the target variable derived from clinical trial outcomes using heuristic rules, unlike the objectively measured features.

The pipeline consists of three main stages:

1. **Data Collection:** We retrieve disease identifiers from MeSH, drug properties and indications from ChEMBL, disease–target associations from Open Targets, clinical trial outcomes from ClinicalTrials.gov, and protein pathway mappings from UniProt/Reactome.
2. **Feature Engineering:** Raw data is processed into numerical features. Drug features include molecular descriptors and fingerprints; disease features capture ontological structure and pathway information; drug–disease features measure pathway overlap and semantic similarity.
3. **Prediction:** We train several machine learning models (logistic regression, random forests, gradient boosting, etc.) to predict whether a drug–disease pair will succeed in clinical trials.

The following sections detail each stage: Section 2 describes the data sources and preprocessing, Section 3 presents the machine learning methodology, and Section 4 reports our findings.

2 Data

2.1 Data Sources

Describe where the data comes from, collection methods, and aggregation process.

Among the vast amount of medical, pharmacological and chemical publicly available data, we focused our attention on those being able to provide us information on basic classification of diseases (MeSH), drug chemical descriptors and targets (ChEMBL), disease targets (Open Targets) as well as information on drug-disease indication studies (ClinicalTrials). Table 1 summarizes information on data sources.

Table 1: Description of data sources.

Source	Access Method	Data Retrieved
MeSH RDF	SPARQL endpoint	Disease IDs and ontology
ChEMBL	REST API (Python client with query-style filters)	Drug chemical properties, targets and indications
UniProt	REST API (Python client)	Map target and pathway IDs
Open Targets	GraphQL API	Disease targets
ClinicalTrials.gov	REST API	Trial results and metadata

2.2 Target variable definition

Start with explaining how you got the labels.

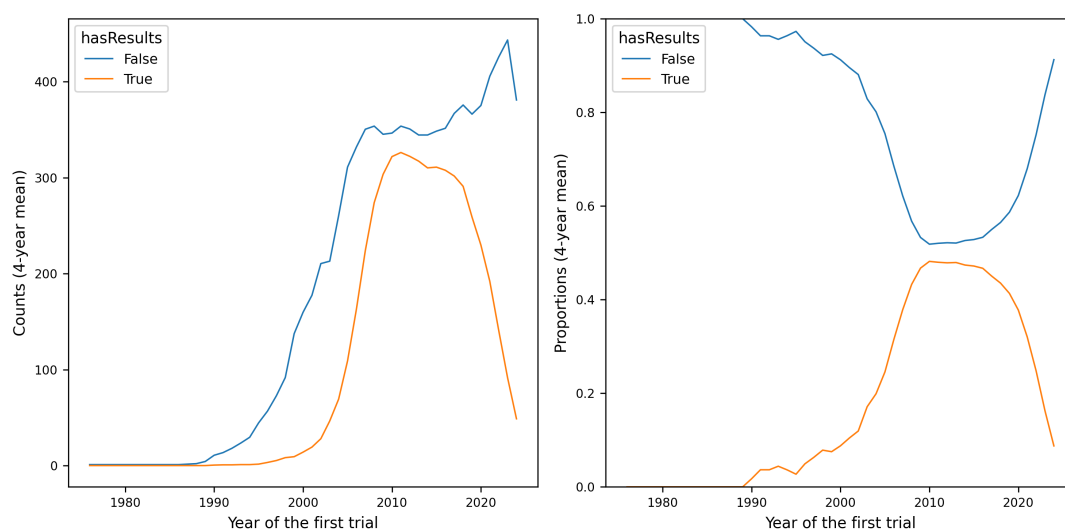


Figure 4: Counts and proportions of published results over the time.

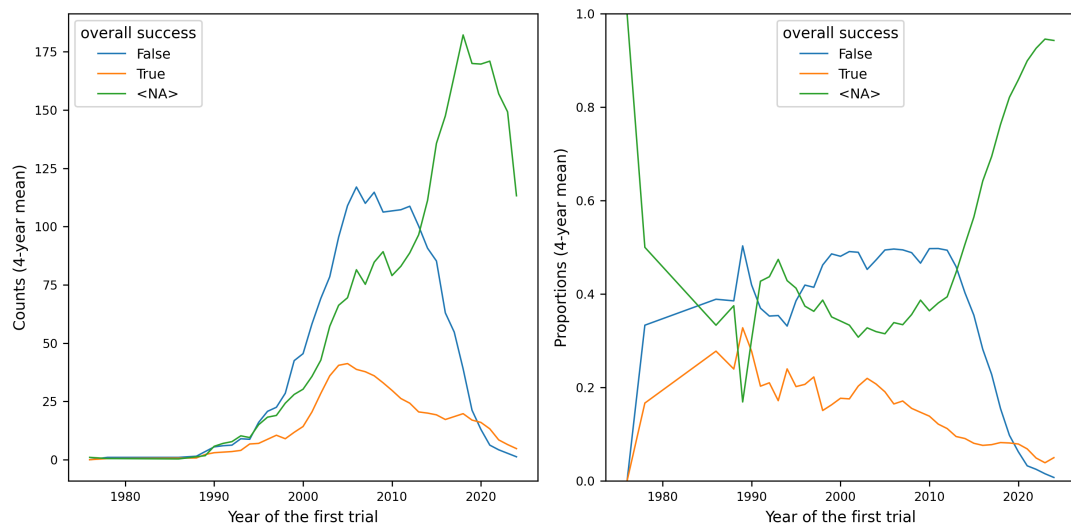


Figure 5: Counts and proportions of label modalities over the time.

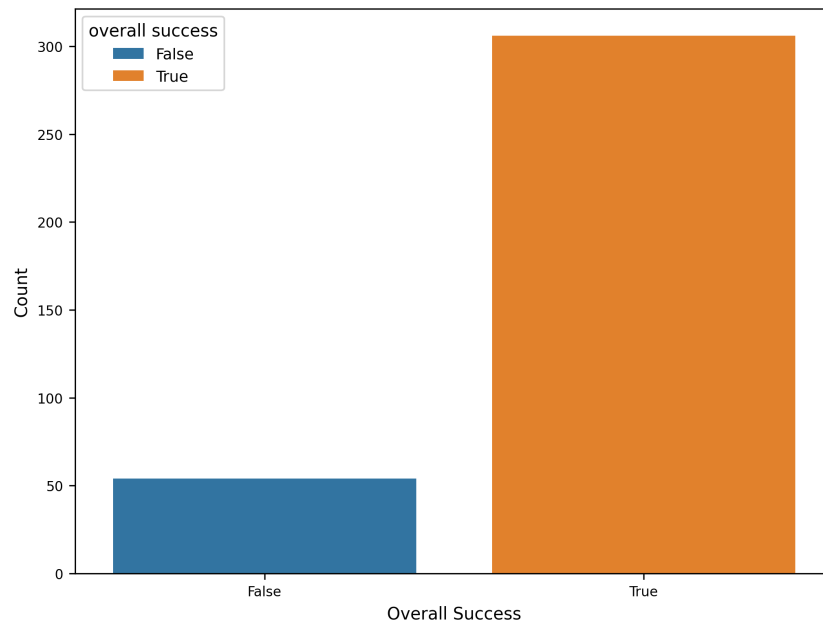


Figure 6: Label distribution for indications with missing date.

2.3 Data Quality and Preprocessing

Discuss missing values, outliers, normalization, feature encoding, or transformations.

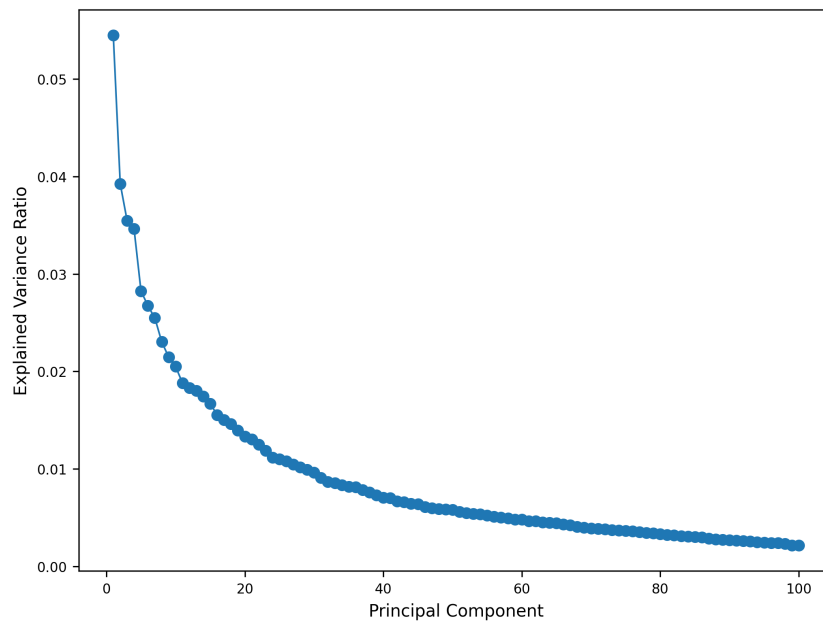


Figure 7: Scree plot of PCA on merged data.

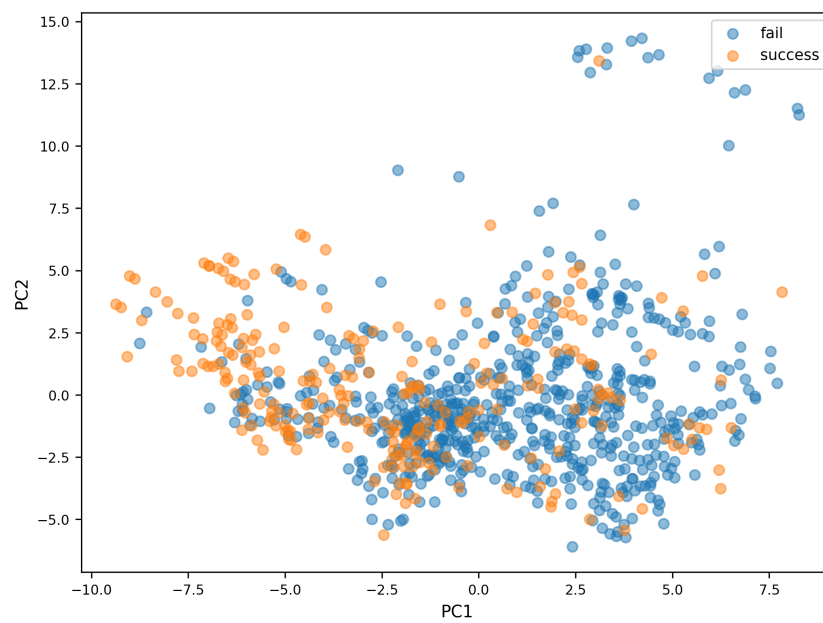


Figure 8: PCA scatter plot.

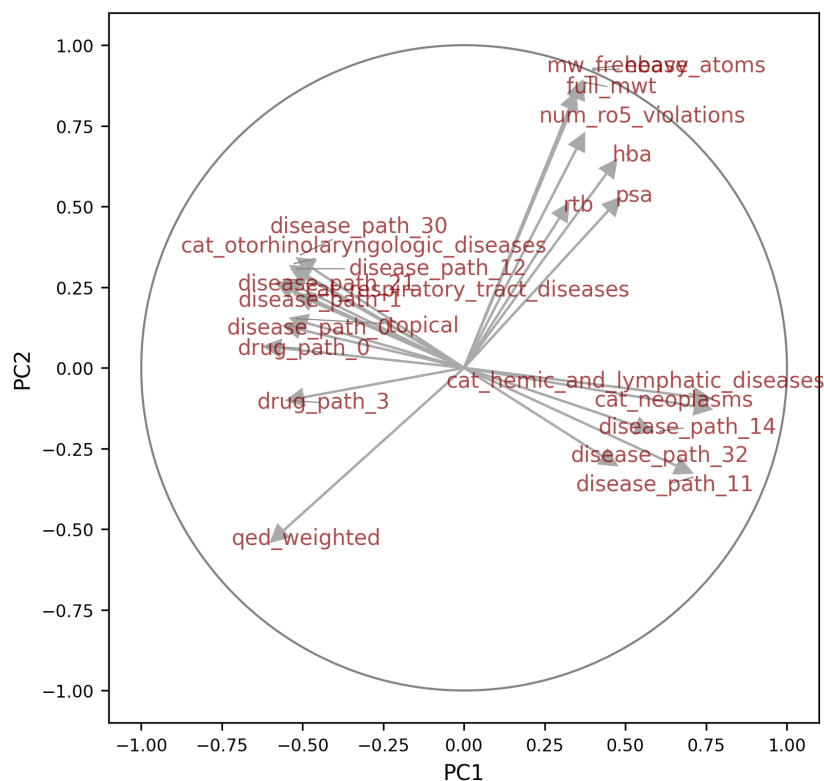


Figure 9: PCA correlation circle.

2.4 Feature Description

Provide a table or list explaining each feature and its meaning. Optionally, include relationships between features.

Table 2: Summary of datasets and merged data.

Dataset	Rows	Columns
Drugs	525	181
Diseases	149	79
Trials	14316	12
Indications	2482	12
Name embeddings	388752	5
Molecular fingerprints	1029	101
Merged data	890	252
Merged data success counts	False: 637, True: 253	

Table 3: Description of features.

Feature	Type	Description
Drug		
drug_id	str	Unique ChEMBL identifier
chemical_probe	bool	Well-characterized and selective, typically used in biological research
dosed_ingredient	bool	Active ingredient that has been dosed in humans
inorganic_flag	bool	Inorganic molecule
natural_product	bool	Derived from a natural source (not fully synthetic)
oral	bool	Known to be administered orally
orphan	bool	Intended for use against a rare condition
parenteral	bool	Known to be administered parenterally
prodrug	bool	Prodrug, activated after administration
therapeutic_flag	bool	Has a therapeutic application, opposed to e.g. imaging
topical	bool	Known to be administered topically
veterinary	bool	Has veterinary application as well
chirality	cat	achiral, single enantiomer, mixture, nan
alogp	float	Octanol-water partition coefficient, lipophilicity indicator
aromatic_rings	int	Number of aromatic rings
full_mwt	float	Molecular weight of the full compound including any salts
hba	float	Number of hydrogen bond acceptors
hbd	float	Number of hydrogen bond donors
heavy_atoms	int	Number of heavy (non-hydrogen) atoms
mw_freebase	float	Molecular weight of parent compound
np_likeness_score	float	Natural product likeness score [Ertl et al., 2008]
num_ro5_violations	int	Violations of Lipinski’s rule-of-five, using HBA and HBD definitions
psa	float	Polar surface area
qed_weighted	float	Weighted quantitative estimate of drug likeness [Bickerton et al., 2012]
ro3_pass	int	Passes the rule-of-three (mw < 300, logP < 3 etc)
rtb	int	Number of rotatable bonds
FP_#	float	First PCs in Morgan molecular fingerprint space
drug_path_#	float	First PCs in TF-IDF space derived from pathway lists
Disease		
disease_id	str	Unique EFO identifier
ontology_depth	int	Depth in disease classification tree
nb_descendants	int	Number of descendants in disease classifications tree
nb_indications	int	Number of drug indications
cat_#	bool	Ontological category other than immune system disorders
disease_path_#	float	First PCs in TF-IDF space derived from pathway lists
Drug–Disease		
success	bool	Consensus success or failure
path_similarity	float	Jaccard similarity of pathway lists
name_similarity	float	Cosine similarity of name embeddings in PubMedBERT (later excluded due to data leak)

```

HeteroData(
  drug={ x=[742, 127] },
  disease={ x=[167, 25] },
  pathway={ x=[1509, 1] },
  (drug, interacts, pathway)={ edge_index=[2, 31266] },

```

```

(disease, associated_with, pathway)={ edge_index=[2, 13608] },
(drug, tried_against, disease)={
  edge_index=[2, 1298],
  edge_label=[1298],
}
)

```

3 Methods

3.1 Model Overview

We frame drug repurposing as a **pairwise prediction problem**: given a drug-disease pair represented by features from both entities, predict whether this combination will be successful. Crucially, this formulation is *symmetric*—neither the drug nor the disease is treated as a fixed query; instead, each (drug, disease) pair constitutes a single data point with features derived from:

- the drug (chemical properties, molecular fingerprints, pathway associations),
- the disease (ontological features, pathway associations), and
- their interaction (e.g., pathway similarity between drug targets and disease-associated pathways).

This symmetric approach differs from conditional formulations such as “given a disease, rank candidate drugs” or “given a drug, identify treatable diseases.” While those asymmetric framings fix one entity and search over the other, our model learns to score arbitrary drug-disease combinations without privileging either entity. The prediction task can be viewed as **link prediction** in a bipartite drug-disease graph, where we predict whether an edge (successful treatment) should exist between a drug node and a disease node.

To address this task, we evaluate six machine learning classifiers spanning different algorithmic paradigms, allowing us to compare the effectiveness of various modeling approaches on our high-dimensional, imbalanced dataset.

3.1.1 Instance-Based Methods

k-Nearest Neighbors (KNN). KNN is a non-parametric method that classifies samples based on the majority class among their k nearest neighbors in feature space. While conceptually simple and making no assumptions about the underlying data distribution, KNN can struggle with high-dimensional data due to the curse of dimensionality. We include it as a baseline to assess whether local similarity in feature space is predictive of treatment success.

3.1.2 Linear Models

Logistic Regression. Logistic regression models the log-odds of the positive class as a linear combination of features, providing interpretable coefficients that quantify each feature’s contribution to the prediction. We employ elastic net regularization, which combines L1 (lasso) and L2 (ridge) penalties, enabling both feature selection and handling

of correlated features. The regularization strength C and the mixing parameter `l1_ratio` are tuned via cross-validation.

Support Vector Machine (SVM). SVMs find an optimal separating hyperplane that maximizes the margin between classes. We explore multiple kernel functions: linear (for linearly separable patterns), polynomial (for moderate non-linear relationships), and radial basis function (RBF) kernels (for complex non-linear boundaries). The regularization parameter C controls the trade-off between margin maximization and classification error, while kernel-specific parameters (degree for polynomial, γ for RBF) modulate the decision boundary complexity.

3.1.3 Ensemble Methods

Random Forest. Random forests aggregate predictions from multiple decision trees, each trained on a bootstrap sample of the data with a random subset of features considered at each split. This bagging approach reduces overfitting and variance compared to individual trees while maintaining the ability to capture non-linear relationships and feature interactions. The ensemble nature also provides natural estimates of feature importance.

Gradient Boosting. Gradient boosting builds an additive model by sequentially fitting weak learners (shallow decision trees) to the residual errors of the current ensemble. Each tree corrects the mistakes of its predecessors, progressively improving predictive accuracy. Key hyperparameters include the number of estimators, tree depth, learning rate (which controls the contribution of each tree), and subsample ratio (for stochastic gradient boosting).

XGBoost. XGBoost (Extreme Gradient Boosting) extends gradient boosting with several enhancements: regularization terms in the objective function to prevent overfitting, efficient handling of sparse data, and parallel tree construction. Additional hyperparameters such as `colsample_bytree` (feature subsampling per tree) and `gamma` (minimum loss reduction for splits) provide finer control over model complexity.

3.1.4 Model Selection Rationale

Our model selection is motivated by several characteristics of the pairwise prediction problem:

- **High dimensionality:** With approximately 250 features (molecular fingerprints, pathway embeddings, chemical descriptors from both drug and disease), we require models that can handle high-dimensional inputs without overfitting. Regularized linear models and tree-based ensembles are well-suited for this setting.
- **Class imbalance:** The dataset exhibits a 2.5:1 ratio of failures to successes. All models are configured with class balancing mechanisms (see Section 3.2).
- **Concatenated pair representations:** Since we represent each drug-disease pair by concatenating drug features, disease features, and interaction features into a single vector, standard tabular classifiers can be applied directly. This contrasts

with graph-based approaches that would model drugs and diseases as separate node types.

- **Interpretability vs. performance trade-off:** Linear models offer interpretability through feature coefficients, while ensemble methods typically achieve higher predictive performance at the cost of interpretability.
- **Mixed feature types:** Our features include continuous chemical descriptors, binary flags, and dimensionality-reduced embeddings. Tree-based methods are naturally suited to mixed feature types, while distance-based methods (KNN, SVM) benefit from proper scaling.

3.2 Implementation Details

3.2.1 Preprocessing Pipeline

All models are embedded in a scikit-learn `Pipeline` that applies consistent preprocessing before classification:

- **Missing value imputation:** Numeric features are imputed with column means; categorical features use the most frequent value. In practice, missing values were largely eliminated during data preparation (see Section 2).
- **Feature scaling:** Numeric features are standardized to zero mean and unit variance using `StandardScaler`, which is essential for distance-based methods (KNN, SVM) and improves convergence for gradient-based optimization (logistic regression).

3.2.2 Handling Class Imbalance

Given the imbalanced class distribution (approximately 71% failures, 29% successes), we employ class weighting strategies:

- For logistic regression, SVM, and random forest, we set `class_weight='balanced'`, which inversely weights classes proportional to their frequencies.
- For XGBoost, we set `scale_pos_weight` to the ratio of negative to positive samples, achieving an equivalent effect.
- KNN and standard gradient boosting do not natively support class weighting; their performance may be affected by the imbalance.

3.2.3 Train-Test Split Strategy

To simulate realistic deployment conditions where models must generalize to future drug-disease pairs, we employ a **temporal split** rather than random stratified sampling. Data are sorted by the first clinical trial date associated with each indication, and the earliest 85% of samples form the training set while the most recent 15% constitute the test set. The split date corresponds to August 2014, ensuring that the model is evaluated on genuinely future data.

This temporal approach prevents data leakage that could occur if training samples contained information from trials conducted after the test set trials, which would artificially inflate performance estimates.

3.2.4 Cross-Validation and Hyperparameter Tuning

Hyperparameter optimization is performed using 5-fold cross-validation on the training set via `GridSearchCV`. The optimization objective is the F1-score for the positive (success) class, chosen because:

- It balances precision and recall, both of which are important: high precision avoids wasting resources on predicted successes that fail, while high recall ensures promising drug-disease pairs are not overlooked.
- F1-score is more informative than accuracy for imbalanced datasets.

Table 4 summarizes the hyperparameter search spaces for each model.

Table 4: Hyperparameter search spaces for each classifier.

Model	Hyperparameters
k-Nearest Neighbors	<code>n_neighbors</code> $\in \{5, 10, 20, 30\}$, <code>weights</code> $\in \{\text{uniform, distance}\}$, <code>p</code> $\in \{1, 2\}$
Logistic Regression	<code>C</code> $\in \{0.01, 0.1, 1, 10, 100\}$, <code>l1_ratio</code> $\in \{0, 0.25, 0.5, 0.75, 1\}$
SVM (linear)	<code>C</code> $\in \{0.01, 0.1, 1, 10\}$
SVM (polynomial)	<code>C</code> $\in \{0.1, 1, 10\}$, <code>degree</code> $\in \{2, 3, 4\}$, <code>gamma</code> $\in \{\text{scale, auto}\}$
SVM (RBF)	<code>C</code> $\in \{0.1, 1, 10\}$, <code>gamma</code> $\in \{\text{scale, auto}\}$
Random Forest	<code>n_estimators</code> $\in \{100, 300, 500\}$, <code>max_depth</code> $\in \{\text{None}, 10, 20\}$, <code>min_samples_split</code> $\in \{2, 5\}$, <code>min_samples_leaf</code> $\in \{1, 2\}$
Gradient Boosting	<code>n_estimators</code> $\in \{100, 300\}$, <code>max_depth</code> $\in \{3, 5\}$, <code>learning_rate</code> $\in \{0.01, 0.05, 0.1\}$, <code>subsample</code> $\in \{0.8, 1.0\}$
XGBoost	<code>n_estimators</code> $\in \{100, 300\}$, <code>max_depth</code> $\in \{3, 5\}$, <code>learning_rate</code> $\in \{0.01, 0.05, 0.1\}$, <code>subsample</code> $\in \{0.8, 1.0\}$, <code>colsample_bytree</code> $\in \{0.8, 1.0\}$, <code>gamma</code> $\in \{0, 0.1\}$

3.2.5 Software and Libraries

All experiments are implemented in Python 3.11 using the following libraries:

- **scikit-learn** (v1.x): Preprocessing, cross-validation, and implementations of KNN, logistic regression, SVM, random forest, and gradient boosting.
- **XGBoost**: High-performance gradient boosting implementation.
- **pandas** and **NumPy**: Data manipulation and numerical operations.
- **RDKit**: Molecular fingerprint generation (Morgan fingerprints with radius 2).
- **Sentence Transformers** (PubMedBERT): Drug and disease name embeddings.

3.2.6 Evaluation Metrics

Model performance is assessed using multiple metrics computed on the held-out test set:

- **F1-score**: Harmonic mean of precision and recall for the positive class.
- **ROC-AUC**: Area under the receiver operating characteristic curve, measuring discrimination ability.

- **Precision-Recall AUC:** More informative than ROC-AUC for imbalanced datasets.
- **Confusion matrix:** Provides a detailed breakdown of true/false positives/negatives.

The best model from cross-validation is refitted on the entire training set before final evaluation on the test set.

4 Results

4.1 Model Comparison

Compare models quantitatively (accuracy, RMSE, AUC, etc.) and qualitatively (strengths/weaknesses).

Table 5: Model comparison summary.

	Model	Mean F1 score	Std F1 score
10	XGBoost	0.642752	0.055461
1	Logistic Regression	0.624640	0.030290
6	Support Vector Machine	0.619498	0.034367
9	Gradient Boost	0.614049	0.068283
8	Random Forest	0.591510	0.035534
0	k-Nearest Neighbors	0.581064	0.064035

Table 6: Best parameters.

	Model	Best parameters
10	XGBoost	colsample_bytree = 0.8, gamma = 0, learning_rate = 0.1, max_depth = 5, n_estimators = 100, subsample = 1.0
1	Logistic Regression	C = 0.1, l1_ratio = 0
6	Support Vector Machine	C = 1, degree = 2, gamma = scale, kernel = poly
9	Gradient Boost	learning_rate = 0.1, max_depth = 5, n_estimators = 300, subsample = 0.8
8	Random Forest	max_depth = 10, min_samples_leaf = 2, min_samples_split = 2, n_estimators = 300
0	k-Nearest Neighbors	n_neighbors = 5, p = 2, weights = distance

4.2 Feature Analysis

Analyze the impact of key features on model performance (e.g., feature importance, correlation).

4.3 Failure and Success Cases

Highlight scenarios where the models perform well or poorly. Include illustrative figures or tables.

5 Discussion

Interpret the results. Relate findings to your original objectives. Discuss limitations and possible improvements.

6 Conclusion

Summarize your work, key findings, and potential future directions.

References

- [Bickerton et al., 2012] Bickerton, G. R., Paolini, G. V., Besnard, J., Muresan, S., and Hopkins, A. L. (2012). Quantifying the chemical beauty of drugs. *Nat Chem*, 4(2):90–98.
- [Ertl et al., 2008] Ertl, P., Roggo, S., and Schuffenhauer, A. (2008). Natural product-likeness score and its application for prioritization of compound libraries. *J. Chem. Inf. Model.*, 48(1):68–74. Epub 2007 Nov 23.